

Short Answer Test – Detailed Answers (Simple & Normal Format)

1. Smart Banking System

(a) Explain the following Object-Oriented Concepts

Class

A **class** is a blueprint or template used to create objects. It defines the attributes (data) and methods (functions) of an object.

Example:

A **BankAccount** class contains Account Number, Customer Name, Balance, Deposit(), Withdraw(), and Transfer() methods.

Object

An **object** is an instance of a class. It contains actual values and performs the functions defined in the class.

Example:

John's Bank Account with Account Number 101 and a balance of ₹50,000 is an object of the BankAccount class.

Message Passing

Message passing is the process by which one object communicates with another object by calling its methods.

Example:

When a customer selects **Transfer Money**, the Customer object sends a request to the BankAccount object to execute the transfer.

Encapsulation

Encapsulation is the process of combining data and methods into a single unit while protecting the data from unauthorized access. Sensitive information is hidden using private access and can only be accessed through public methods.

Example:

The account balance is private and can only be modified using Deposit() and Withdraw() methods.

Advantages:

- Protects data from unauthorized access.
- Improves security.
- Prevents accidental modification of data.
- Makes the program easier to maintain.

Polymorphism

Polymorphism means one method can perform different tasks depending on the object or situation. It allows the same interface to be used for different implementations.

Example:

The Payment() method behaves differently for UPI Payment, Debit Card Payment, and Net Banking Payment.

(b) Object Relationships**Association**

Association is a relationship where two objects work together but remain independent of each other.

Example:

A Customer has a Bank Account. Both objects can exist independently.

Aggregation

Aggregation is a "Has-A" relationship where the child object can exist without the parent object.

Example:

A Bank has many Customers. Even if the bank closes, customer records may still exist.

Composition

Composition is a strong "Has-A" relationship where the child object cannot exist without the parent object.

Example:

A Bank Account contains Transaction History. If the bank account is deleted, its transaction history is also deleted.

(c) How OOP Improves Banking Software

Object-oriented programming makes banking software more efficient and secure. It improves code reusability by allowing developers to reuse existing classes. Encapsulation protects customer information from unauthorized access. Polymorphism makes it easier to add new payment methods without changing existing code. The modular structure simplifies maintenance, reduces duplicate code, and makes future enhancements such as loans, insurance, and investment services easier to implement.

2. Smart Healthcare Management System**(a) Object-Oriented Design Principles****Abstraction**

Abstraction hides unnecessary implementation details and shows only the essential features to the user. This makes the system simple and user-friendly.

Example:

A patient clicks the **Book Appointment** button without knowing how the system checks doctor availability and stores appointment details.

Encapsulation

Encapsulation protects patient information by allowing access only through authorized methods. Medical records remain secure and confidential.

Modularity

Modularity divides the software into smaller independent modules, making development and maintenance easier.

Example:

- Patient Module
- Doctor Module
- Appointment Module
- Billing Module
- Pharmacy Module

Each module performs a specific task independently.

(b) SOLID Principles

Liskov Substitution Principle (LSP)

The Liskov Substitution Principle states that a child class should be able to replace its parent class without affecting the correctness of the program.

Example:

A SpecialistDoctor or GeneralDoctor object can be used wherever a Doctor object is expected.

Interface Segregation Principle (ISP)

The Interface Segregation Principle states that a class should not be forced to implement methods that it does not need.

Example:

Separate interfaces are created for Patients, Doctors, and Administrators so that each user accesses only relevant functions.

(c) Benefits

These design principles improve software quality by making the system flexible, secure, and easy to maintain. They reduce dependency between modules, encourage code reuse, simplify future updates, and improve overall software performance.

3. Online Food Delivery System

(a) SDLC Phases

Planning

Planning is the first phase of the Software Development Life Cycle. During this phase, the project objectives, budget, timeline, and required resources are identified.

Requirement Analysis

In this phase, developers collect and analyze customer requirements.

Example:

- User Registration
- Restaurant Search

- Food Ordering
- Online Payment
- Order Tracking

Design

The system architecture, database design, user interface, and software structure are prepared before coding begins.

Development

Developers write the program using suitable programming languages and implement all required features.

Testing

The software is tested to identify and fix errors before deployment. Different testing methods such as Unit Testing, Integration Testing, and System Testing are performed.

Maintenance

After deployment, bugs are fixed, security updates are applied, and new features are added according to customer requirements.

(b) Purpose of Each Phase

Phase	Purpose
Planning	Defines project goals, budget, and schedule.
Requirement Analysis	Collects and analyzes user requirements.
Design	Creates system architecture and database design.
Development	Builds the software by writing code.
Testing	Detects and fixes software defects.
Maintenance	Updates and improves the software after deployment.

(c) Importance of SDLC

The Software Development Life Cycle ensures that software is developed in a systematic manner. It improves software quality, reduces development cost, minimizes project risks, ensures timely delivery, and increases customer satisfaction by delivering reliable software.

4. Smart Agriculture Monitoring System

(a) SDLC Models

Prototype Model

The Prototype Model develops a sample version of the software before building the final product. Customer feedback is collected to improve the system.

Spiral Model

The Spiral Model combines development with continuous risk analysis. The project is developed through multiple cycles until the final product is completed.

Agile Model

The Agile Model develops software in small iterations. Customer feedback is collected after every iteration, making it easy to modify requirements.

DevOps Model

The DevOps Model combines development and operations teams to automate software development, testing, deployment, and maintenance using Continuous Integration and Continuous Delivery (CI/CD).

(b) Prototype Model vs Agile Model

Prototype Model	Agile Model
Develops a sample first	Develops software in iterations
Focuses on understanding requirements	Focuses on continuous improvement
Feedback before development	Feedback throughout development
Suitable for unclear requirements	Suitable for changing requirements

(c) Recommended Model

The **Agile Model** is the most suitable model because customer requirements change frequently. Agile supports continuous customer feedback, faster software delivery, easy modification of features, improved software quality, and greater customer satisfaction.

5. Smart Transport Management System

(a) Importance of UML

Unified Modeling Language (UML) is a standard language used to visualize, design, and document software systems before development begins. It helps developers understand the system structure and behavior.

Advantages:

- Improves communication among stakeholders.
- Reduces development errors.
- Provides proper documentation.
- Makes software easier to understand.
- Simplifies maintenance.

(b) UML Diagrams

State Diagram

A State Diagram shows the different states of an object during its life cycle.

Example:

Bus → Available → Running → Maintenance → Completed

Component Diagram

A Component Diagram shows the major software modules and their relationships.

Example:

- Login Module
- Booking Module
- Payment Module
- Notification Module

Deployment Diagram

A Deployment Diagram shows how software components are deployed on hardware devices.

Example:

Client → Web Server → Application Server → Database Server

Package Diagram

A Package Diagram groups related classes into packages to organize the software.

Example:

- User Package
- Booking Package
- Payment Package
- Admin Package

(c) Benefits of UML

UML improves communication among developers, analysts, and customers. It provides proper documentation, reduces misunderstandings, simplifies software maintenance, and helps in designing high-quality software systems.

6. Smart Inventory Management System**(a) Modular Software Design**

Modular software design divides a large software system into smaller independent modules. Each module performs a specific function and can be developed, tested, and maintained independently.

Example:

- Product Module
- Supplier Module
- Stock Module
- Billing Module
- Report Module

(b) Procedural Programming vs Object-Oriented Programming

Procedural Programming	Object-Oriented Programming
Uses functions	Uses classes and objects
Data and functions are separate	Data and methods are combined
Less secure	More secure
Less reusable	Highly reusable
Harder to maintain	Easier to maintain
Example: C	Examples: Java, C++, Python

(c) Benefits of Modularity

Modularity improves software scalability by allowing new modules to be added easily as the system grows. It improves maintainability because errors can be fixed within a single module without affecting the entire system. Reusability allows existing modules to be used in other software projects, reducing development time and cost. Overall, modular software design simplifies testing, encourages teamwork, improves software quality, and makes future enhancements easier.